Лабораторная работа 7 «Настройка гиперпараметров моделей машинного обучения»

В этой лабораторной работе описывается, как с помощью генетических алгоритмов улучшить качество моделей машинного обучения с учителем за счет настройки их гиперпараметров. Мы начнем с краткого введения в настройку гиперпараметров, а затем опишем идею поиска на сетке. После описания набора данных о винах Wine и классификатора на основе адаптивного усиления (то и другое используется в этой лабораторной работе) мы продемонстрируем настройку гиперпараметров как с помощью традиционного поиска на сетке, так и путем применения к поиску генетического алгоритма. Наконец, попробуем улучшить результаты, применив генетический алгоритм напрямую.

В этой лабораторной работе мы:

- узнаем о том, что такое настройка гиперпараметров в машинном обучении;
- познакомимся с набором данных Wine и классификатором на основе адаптивного усиления;
- улучшим качество классификатора с помощью поиска гиперпараметров на сетке;
- улучшим качество классификатора с помощью поиска на сетке, управляемого генетическим алгоритмом;
- улучшим качество классификатора, непосредственно применив генетический алгоритм к настройке гиперпараметров.

Глава начинается кратким обзором роли гиперпараметров в машинном обучении.

Технические требования

В этой главе нам понадобится Python 3 и следующие библиотеки:

- `deap`;
- `numpy`;
- `pandas`;
- `matplotlib`;
- `seaborn`;
- `sklearn`;
- `sklearn-deap` – описывается здесь же.

Кроме того, мы будем использовать набор данных Wine Калифорнийского университета в Ирвайне (https://archive.ics.uci.edu/ml/datasets/Wine).

Код приведенных в этой главе программ можно найти в репозитории книги на GitHub по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/tree/master/Chapter08. Чтобы увидеть код в действии, посмотрите видео по адресу http://bit.ly/37Q45id.

Гиперпараметры в машинном обучении

В главе 7 мы описывали машинное обучение как процесс программной корректировки (или настройки) внутренних параметров модели, так чтобы она порождала желаемые выходы в ответ на подаваемые входные данные. Для этого с каждой моделью обучения с учителем ассоциируется алгоритм обучения, который итеративно корректирует свои внутренние параметры на этапе обучения.

Однако у большинства моделей есть еще один набор параметров, которые задаются **до** начала обучения. Они называются **гиперпараметрами** и влияют на сам способ обучения. На рисунке ниже показаны оба типа параметров.



Настройка гиперпараметров модели машинного обучения

Обычно у гиперпараметров имеются значения по умолчанию, которые применяются, если никакие другие не заданы явно. Например, заглянув в реализацию решающего дерева в библиотеке `sklearn` (https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html), мы увидим несколько гиперпараметров вместе с их значениями по умолчанию. Некоторые из них приведены в таблице ниже.

Имя	Тип	Описание	Значение по умолчанию
`max_depth`	int	Максимальная глубина дерева	Нет
`splitter`	string	Стратегия разделения в каждом узле (`best` или `random`)	`best`
`min_samples_split`	int или float	Минимальное количество примеров, при котором производится разделение	2

Каждый из этих параметров влияет на процесс построения дерева на этапе обучения, а их совокупное воздействие на результат обучения и, следовательно, на качество модели может быть весьма существенно.

Поскольку выбор гиперпараметров может оказать заметный эффект на качество моделей машинного обучения, специалисты по обработке данных часто тратят много времени на поиск лучших комбинаций гиперпараметров. Этот процесс и называется **настройкой гиперпараметров**. В следующем подразделе описаны некоторые методы настройки гиперпараметров.

Настройка гиперпараметров

Широко распространенный способ отыскания хороших комбинаций гиперпараметров – **поиск на сетке**. При использовании этого метода мы выбираем подмножество значений каждого гиперпараметра, подлежащего настройке. Например, в случае решающего дерева можно выбрать подмножество {2, 5, 10} значений гиперпараметра `max_depth`, а для `splitter` оставить оба допустимых значения – `{"best", "random"}`. Тогда нужно будет исследовать все шесть комбинаций этих значений. Для каждой комбинации классификатор обучается и оценивается по некоторому критерию, например верности. В итоге мы выбираем ту комбинацию, которая дала наилучшее качество.

Основной недостаток поиска на сетке – необходимость полного перебора всех комбинаций, что может потребовать очень много времени. Сократить время позволяет **случайный поиск**, когда выбираются и проверяются случайные комбинации гиперпараметров.

Нам интереснее вариант поиска на сетке, при котором для нахождения наилучших комбинаций применяется генетический алгоритм. Он открывает возможность отыскать лучшую комбинацию за время меньшее, чем в случае исчерпывающего поиска на сетке.

Поиск на сетке и случайный поиск поддерживаются библиотекой `sklearn`, но для применения генетического алгоритма нужна библиотека `sklearn-deap`, построенная на базе каркаса DEAP. Для ее установки выполните команду

```
pip install sklearn-deap
```

В следующих разделах мы протестируем и сравним обе версии поиска на сетке – полным перебором и с применением генетического алгоритма. Но сначала познакомимся с набором данных Wine, на котором будем экспериментировать.

Набор данных Wine

Широко известен набор данных о винах Wine из репозитория машинного обучения Калифорнийского университета в Ирвайне. Он находится по адресу https://archive.ics.uci.edu/ml/datasets/Wine и содержит результаты химического анализа 178 разных вин, произведенных в одном и том же регионе Италии. Каждому вину присвоена одна из трех категорий.

Химический анализ производится по 13 показателям, представляющим следующие характеристики вина:

- процентное содержание алкоголя;
- содержание яблочной кислоты;
- содержание золы;
- содержание щелочи;
- содержание магния;
- общее количество фенолов;
- флаваноидные фенолы;
- нефлаваноидные фенолы;
- проантоцианиды;
- интенсивность цвета;
- оттенок вина;
- разбавленность вина;
- сорт вина.

В столбцах 2–4 набора данных содержатся результаты измерений, а результат классификации – сам тип вина (1, 2 или 3) – находится в первом столбце.

Теперь рассмотрим классификатор, который мы собираемся использовать.

Классификатор на основе адаптивного усиления

Алгоритм адаптивного усиления, или **AdaBoost**, – эффективная модель машинного обучения, которая объединяет результаты нескольких простых алгоритмов обучения (слабых обучаемых), вычисляя их взвешенную сумму. AdaBoost складывает экземпляры слабых обучаемых в процессе обучения, корректируя их с целью улучшить результаты для входных примеров, которые до того были классифицированы неверно.

В реализации этой модели в библиотеке `sklearn`, AdaboostClassifier (https://scikitlearn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html), имеется несколько гиперпараметров, некоторые приведены в следующей таблице.

Имя	Тип	Описание	Значение по умолчанию
n_estimators	int	Максимальное число оценивателей	50
learning_rate	float	Позволяет уменьшить вклад каждого классификатора	1
algorithm	{'SAMME', 'SAMME.R'}	'SAMME' – используется вещественный алгоритм усиления; 'SAMME.R' – используется дискретный алгоритм усиления	2

Интересно, что типы всех трех гиперпараметров различны – int, float и перечисление. Ниже мы узнаем, как методы настройки обращаются с этими типами. В следующем разделе описываются две формы поиска на сетке.

Настройка гиперпараметров с помощью генетического поиска на сетке

Чтобы инкапсулировать настройку гиперпараметров классификатора AdaBoost для набора данных Wine с помощью поиска на сетке – традиционного и управляемого генетическим алгоритмом, – мы создали Python-класс HyperparameterTuningGrid, который находится в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter08/01-hyperparameter-tuning-grid.py.

Ниже описаны основные части этого класса.

1. Метод класса __init__() инициализирует набор данных, классификатор AdaBoost, *k*-групповой перекрестный контроль и параметры сетки:

```
self.initWineDataset()
self.initClassifier()
self.initKfold()
self.initGridParams()
```

2. Метод initGridParams() инициализирует поиск на сетке, задавая множество значений трех гиперпараметров, упомянутых в предыдущем разделе:
   - для параметра n_estimators проверяется 10 значений, равномерно распределенных между 10 и 100;
   - для параметра learning_rate проверяется 10 значений, логарифмически равномерно распределенных между 0.1 (10^{-2}) и 1 (10^{0});
   - для параметра algorithm проверяются оба допустимых значения: 'SAMME' и 'SAMME.R'.

   Таким образом, нам предстоит проверить 200 (10×10×2) различных комбинаций параметров:

```
self.gridParams = {
    'n_estimators': [10, 20, 30, 40, 50, 60, 70, 80, 90, 100],
```

```
    'learning_rate': np.logspace(-2, 0, num=10, base=10),
    'algorithm': ['SAMME', 'SAMME.R'],
}
```

3. Метод `getDefaultAccuracy()` оценивает верность классификатора с подразумеваемыми по умолчанию значениями гиперпараметров:

```
cv_results = model_selection.cross_val_score(self.classifier,
                                             self.X,
                                             self.y,
                                             cv=self.kfold,
                                             scoring='accuracy')
return cv_results.mean()
```

4. Метод `gridTest()` выполняет традиционный поиск на определенной ранее сетке значений гиперпараметров. Применяя *k*-групповой перекрестный контроль и вычисляя среднюю верность, он находит лучшую комбинацию значений:

```
gridSearch = GridSearchCV(estimator=self.classifier,
                          param_grid=self.gridParams,
                          cv=self.kfold,
                          scoring='accuracy')

gridSearch.fit(self.X, self.y)
```

5. Метод `geneticGridTest()` выполняет поиск на сетке, управляемый генетическим алгоритмом. Для этого используется метод `EvolutionaryAlgorithmSearchCV()` из библиотеки `sklearn-deap`, специально разработанный так, чтобы вызов был похож на традиционный поиск на сетке. Нужно только добавить несколько параметров генетического алгоритма: размер популяции, вероятность мутации, размер турнира и количество поколений:

```
gridSearch =
EvolutionaryAlgorithmSearchCV(estimator=self.classifier,
                              params=self.gridParams,
                              cv=self.kfold,
                              scoring='accuracy',
                              verbose=True,
                              population_size=20,
                              gene_mutation_prob=0.30,
                              tournament_size=2,
                              generations_number=5)
                              gridSearch.fit(self.X, self.y)
```

6. Наконец, метод `main()` начинает с того, что оценивает качество классификатора с подразумеваемыми по умолчанию значениями гиперпараметров. После этого он прогоняет сначала традиционный полный поиск на сетке, а затем генетический поиск на сетке, замеряя в каждом случае время.

Результаты выполнения метода `main` этого класса описаны в следующем подразделе.

Тестирование качества классификатора с параметрами по умолчанию

Результаты прогона показывают, что при подразумеваемых по умолчанию значениях `n_estimators = 50`, `learning_rate = 1.0` и `algorithm = 'SAMME.R'` верность классификации равна приблизительно 65 %:

Значения гиперпараметров классификатора по умолчанию:

```
{'algorithm': 'SAMME.R', 'base_estimator': None, 'learning_rate': 1.0,
'n_estimators': 50, 'random_state': 42}
Оценка для значений по умолчанию = 0.6457142857142857
```

Эту верность хорошей не назовешь. Будем надеяться, что поиск на сетке позволит улучшить ее, отыскав более подходящую комбинацию гиперпараметров.

Результаты традиционного поиска на сетке

Далее запускается традиционный исчерпывающий поиск на сетке с перебором всех 200 возможных комбинаций. Выясняется, что лучшей является комбинация `n_estimators = 70`, `learning_rate ≈ 0.359`, `algorithm = 'SAMME.R'`.

При этом достигается верность классификации приблизительно 93 % – гораздо лучше, чем первоначальные 65 %. Время поиска составило около 32 секунд:

```
производится поиск на сетке...
лучшие параметры: {'algorithm': 'SAMME.R', 'learning_rate':
0.3593813663804626, 'n_estimators': 70}
лучшая оценка: 0.9325842696629213
затраченное время = 32.180874824523926
```

Далее у нас по плану генетический поиск на сетке. Удастся ли улучшить результаты? Посмотрим.

Результаты генетического поиска на сетке

Последняя часть прогона – генетический поиск на сетке с теми же параметрами. Распечатка начинается с загадочного сообщения:

```
выполняется генетический поиск на сетке...
обнаружены типы [1, 2, 1] и maxint [9, 9, 1]
```

Эти сообщения относятся к сетке, на которой производится поиск: список из 10 целых чисел (значений `n_estimators`), массив `ndarray` из 10 элементов (значений `learning_rate`) и список из двух строк (значений `algorithm`), а именно:

- `'типы [1, 2, 1]'` описывает типы значений: [список, ndarray, список];
- `'maxint [9, 9, 1]'` описывает размеры списков и массивов: [10, 10, 2].

Следующая строка говорит об общем количестве комбинаций параметров (10×10×2):

```
--- Эволюция на 200 возможных комбинациях ---
```

Остаток распечатки нам хорошо знаком, потому что используются те самые реализации генетических алгоритмов в DEAP, с которыми мы работали всю дорогу. Точнее, печатается статистика каждого поколения:

```
gen nevals avg      min      max      std
0   20     0.642135 0.117978 0.904494 0.304928
1   14     0.807865 0.123596 0.91573  0.20498
2   15     0.829775 0.123596 0.921348 0.172647
3   12     0.885393 0.679775 0.921348 0.0506055
4   13     0.903652 0.865169 0.926966 0.0176117
5   11     0.905618 0.797753 0.932584 0.027728
```

В конце работы печатается лучшая найденная комбинация, а также оценка и затраченное время:

```
Лучший индивидуум: {'n_estimators': 70, 'learning_rate':
0.3593813663804626, 'algorithm': 'SAMME.R'}
приспособленность: 0.9325842696629213
затраченное время = 10.997037649154663
```

Эти результаты показывают, что генетический поиск на сетке смог найти тот же лучший результат, что и исчерпывающий поиск, но за меньшее время – примерно 11 секунд.

Отметим, что это простой пример, в котором алгоритм работает очень быстро. На практике часто встречаются куда большие наборы данных, а также сложные модели и многомерные сетки гиперпараметров. В таких случаях исчерпывающий поиск на сетке может занять недопустимо много времени, тогда как генетический поиск способен дать хорошие результаты за приемлемое время.

Тем не менее любой поиск на сетке ограничен определенным нами подмножеством гиперпараметров. А что, если мы хотим произвести поиск вне сетки, не ограничиваясь заранее заданным подмножеством? Возможное решение описано в следующем разделе.

Прямой генетический подход к настройке гиперпараметров

Генетические алгоритмы можно использовать не только как эффективный способ поиска на сетке, но и для прямого поиска во всем пространстве параметров – так же, как мы делали во многих задачах, рассмотренных в этой

книге. Каждый гиперпараметр можно представить переменной, участвующей в поиске, а хромосомой тогда будет комбинация всех таких переменных.

Поскольку типы гиперпараметров могут быть различны, например float, int и перечисление, как в классификаторе AdaBoost, возможно, придется кодировать их по-разному, а в качестве генетических операций использовать какую-то комбинацию отдельных операторов, адаптированных к конкретным типам. Но можно и не «заморачиваться», а представить все параметры числами с плавающей точкой и тем самым упростить алгоритм, что мы и сделаем далее.

Представление гиперпараметров

В главе 6 мы использовали генетические алгоритмы для оптимизации функций вещественных параметров. Параметры были представлены списками чисел с плавающей точкой, например:

```
[1.23, 7.2134, -25.309]
```

Поэтому и генетические операторы были специализированы для работы с такими списками.

Чтобы адаптировать этот подход к настройке гиперпараметров, мы будем представлять каждый гиперпараметр числом с плавающей точкой независимо от его истинного типа. Нужно только придумать, как преобразовать параметр в число с плавающей точкой, а затем вернуться к исходному представлению. Мы поступим следующим образом:

- `n_estimators`, целое число, будет представлено числом типа float в некотором диапазоне, например [1, 100]. Для преобразования float в int мы воспользуемся функцией Python `round()`, которая округляет вещественное число до ближайшего целого;
- параметр `learning_rate` уже имеет тип float, так что никакого преобразования не нужно. Он находится в диапазоне [0.01, 1.0];
- параметр `algorithm` может принимать одно из двух значений: `'SAMME'` и `'SAMME.R'`. Мы будем представлять его числом типа float в диапазоне [0, 1]. Для преобразования обратно в целое округлим до ближайшего целого – 0 или 1, а затем заменим 0 на `'SAMME'`, а 1 – на `'SAMME.R'`.

Код этих преобразований находится в двух Python-файлах, которые описаны в следующих подразделах.

Оценка верности классификатора

Оценка верности классификатора инкапсулирована в Python-классе `HyperparameterTuningGenetic`, находящемся в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter08/hyperparameter_tuning_genetic_test.py.

Ниже описаны основные функции этого класса.

1. Метод класса `convertParam()` принимает список `params`, содержащий значения гиперпараметров типа float, и преобразует их в значения истинных типов, как было описано в предыдущем подразделе:

```
n_estimators = round(params[0])
learning_rate = params[1]
algorithm = ['SAMME', 'SAMME.R'][round(params[2])]
```

2. Метод `getAccuracy()` принимает список чисел типа float, представляющих значения гиперпараметров, вызывает метод `convertParam()` для преобразования их в истинные значения и инициализирует классификатор `ADABooost` этими значениями:

```
n_estimators, learning_rate, algorithm = self.convertParams(params)
self.classifier = AdaBoostClassifier(n_estimators=n_estimators,
                                      learning_rate=learning_rate,
                                      algorithm=algorithm)
```

3. Затем он вычисляет верность классификатора, используя *k*-групповой перекрестный контроль, как в наборе данных Wine:

```
cv_results = model_selection.cross_val_score(self.classifier,
                                             self.X,
                                             self.y,
                                             cv=self.kfold,
                                             scoring='accuracy')
return cv_results.mean()
```

Этот класс применяется в программе, реализующей генетический алгоритм настройки гиперпараметров, которая описана в следующем подразделе.

Настройка гиперпараметров с помощью генетического алгоритма

Генетический поиск лучших гиперпараметров реализован в программе, находящейся в файле https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter08/02-hyperparameter-tuning-genetic.py.

Ниже описаны основные части программы.

1. Задаем нижние и верхние границы всех чисел типа float, представляющих гиперпараметры: [1, 100] для `n_estimators`, [0.01, 1] для `learning_rate` и [0, 1] для `algorithm`:

```
# [n_estimators, learning_rate, algorithm]:
BOUNDS_LOW = [ 1, 0.01, 0]
BOUNDS_HIGH = [100, 1.00, 1]
```

2. Создаем экземпляр класса `HyperparameterTuningGenetic`, который позволит нам проверить различные комбинации гиперпараметров:

```
test = hyperparameter_tuning_genetic.HyperparameterTuningGenetic(RANDOM_SEED)
```

3. Поскольку мы стремимся максимизировать верность классификатора, определим единственную цель – максимизирующую стратегию приспособления:

```
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
```

4. Вот теперь мы подошли к интересной части: т. к. решение представлено списком чисел типа float, принадлежащих различным диапазонам, мы в цикле обойдем все пары, состоящие из нижней и верхней границ, и для каждого гиперпараметра создадим в инструментарии свой оператор, который будет генерировать случайное число в соответствующем диапазоне:

```
for i in range(NUM_OF_PARAMS):
    # "hyperparameter_0", "hyperparameter_1", ...
    toolbox.register("hyperparameter_" + str(i),
                     random.uniform,
                     BOUNDS_LOW[i],
                     BOUNDS_HIGH[i])
```

5. Затем создаем кортеж `hyperparameters`, содержащий только что созданные генераторы случайных чисел для отдельных гиперпараметров:

```
hyperparameters = ()
for i in range(NUM_OF_PARAMS):
    hyperparameters = hyperparameters + \
(toolbox.__getattribute__("hyperparameter_" + str(i)),)
```

6. Теперь мы можем использовать этот кортеж в сочетании со встроенным в DEAP оператором `initCycle()`, чтобы создать новый оператор `individualCreator`, который инициализирует индивидуум комбинацией случайных значений гиперпараметров:

```
toolbox.register("individualCreator",
tools.initCycle,
creator.Individual,
hyperparameters,
n=1)
```

7. Далее мы просим генетический алгоритм использовать метод `getAccuracy()` экземпляра `HyperparameterTuningGenetic` для вычисления приспособленности. Напомним, что метод `getAccuracy()`, описанный в предыдущем подразделе, преобразует переданного ему индивидуума – список трех чисел типа float – в значения соответствующих гиперпараметров, обучает классификатор с этими значениями и вычисляет верность, пользуясь *k*-групповым перекрестным контролем:

```
def classificationAccuracy(individual):
    return test.getAccuracy(individual),

toolbox.register("evaluate", classificationAccuracy)
```

8. Осталось определить генетические операторы. В качестве оператора отбора используем турнир размера 2, а для скрещивания и мутации возьмем операторы, специализированные для хромосом в виде списков чисел с плавающей точкой, ограниченных предельными значениями гиперпараметров:

```
toolbox.register("select", tools.selTournament, tournsize=2)

toolbox.register("mate",
                 tools.cxSimulatedBinaryBounded,
                 low=BOUNDS_LOW,
                 up=BOUNDS_HIGH,
                 eta=CROWDING_FACTOR)

toolbox.register("mutate",
                 tools.mutPolynomialBounded,
                 low=BOUNDS_LOW,
                 up=BOUNDS_HIGH,
                 eta=CROWDING_FACTOR,
                 indpb=1.0 / NUM_OF_PARAMS)
```

9. Как и раньше, будем использовать элитистский подход, т. е. без изменения копировать лучших на данный момент индивидуумов из зала славы в следующее поколение:

```
population, logbook = elitism.eaSimpleWithElitism(population,
                                                   toolbox,
                                                   cxpb=P_CROSSOVER,
                                                   mutpb=P_MUTATION,
                                                   ngen=MAX_GENERATIONS,
                                                   stats=stats,
                                                   halloffame=hof,
                                                   verbose=True)
```

После прогона алгоритма для 5 поколений с размером популяции 20 получается такой результат:

```
gen nevals max      avg
0   20     0.92127  0.841024
1   14     0.943651 0.900603
2   13     0.943651 0.912841
3   14     0.943651 0.922476
4   15     0.949206 0.929754
5   13     0.949206 0.938563
- Лучшее решение:
params = 'n_estimators'= 69, 'learning_rate'=0.628, 'algorithm'=SAMME.R
Верность = 0.94921
```

Таким образом, лучшая найденная комбинация имеет вид `n_estimators = 69`, `learning_rate = 0.628`, `algorithm = 'SAMME.R'`. Достигнутая при таких значениях верность равна приблизительно 94.9 % – заметное улучшение по сравнению с верностью, достигнутой поиском на сетке. Интересно, что лучшие значе-

ния n_estimators и learning_rate оказались вне той сетки, на которой мы их искали.

Резюме

В этой главе мы занимались настройкой гиперпараметров в машинном обучении. Познакомившись с набором данных Wine и классификатором с адаптивным усилением AdaBoost, мы затем представили методы настройки гиперпараметров посредством исчерпывающего поиска на сетке и его генетического аналога. Мы сравнили оба метода и напоследок испытали подход на основе прямого применения генетического алгоритма, когда все гиперпараметры были представлены числами с плавающей точкой. Этот подход позволил улучшить результаты поиска на сетке.

Задание для самостоятельной работы.

1. Изучить материалы лабораторной работы.

2. Ознакомиться с дополнительными источниками.

- библиотека sklearn-deap на GitHub: https://github.com/rsteca/sklearn-deap;
- сравнение EvolutionaryAlgorithmSearchCV с GridSearchCV и RandomizedSearchCV: https://github.com/rsteca/sklearn-deap/blob/master/test.ipynb;
- классификатор ADABoost в библиотеке sklearn: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html;